

TechSaúde — Relatório Técnico de Desenvolvimento: Tela de Verificação de Documento

FATEC Barueri — Gestão de TI | Escopo: front-end de verificacao_documento.html (sem banco de dados/API)

1. Stack e arquitetura

Arquivo único e autocontido (`verificacao_documento.html`), sem build step, sem dependências de terceiros além de fontes via CDN (Google Fonts: Poppins e Inter). Reaproveita o design system das demais telas (paleta creme/verde, tipografia, cartão central arredondado), alternando internamente entre painéis de fundo claro (seleção e confirmação) e painéis de fundo escuro (captura de frente/verso), replicando o contraste já usado nas referências de câmera fornecidas. Divisão interna:

- **HTML5 ~239 linhas** — marcação dividida em quatro blocos de etapa (`#screen-select`, `#screen-front`, `#screen-back`, `#screen-confirm`), cada etapa de captura repetindo a mesma estrutura (viewport de câmera + dropzone de upload + controles), permitindo reaproveitar a mesma lógica de JS para frente e verso.
- **CSS3 ~234 linhas** — variáveis herdadas do mesmo `:root` das outras telas, mais classes específicas do fluxo de captura (`.mode-toggle`, `.viewport-wrap`, `.corner`, `.dropzone`, `.review-row`, `.step-dots`).
- **JavaScript ~195 linhas** — vanilla JS, sem dependências externas; controla a máquina de estados do fluxo (`goTo()`), o acesso à câmera (`getUserMedia`) e a leitura de arquivos (`FileReader`), através de uma função genérica `initCapture(side)` reaproveitada para as etapas de frente e de verso.

2. Histórico de iterações

- **v1** — Levantamento de requisitos por elicitação direta com o solicitante antes da implementação: definição de que o fluxo deveria cobrir (a) escolha do tipo de documento, (b) captura de frente e verso, e (c) confirmação; e de que a captura deveria aceitar **tanto foto pela câmera quanto anexo de arquivo**, e não apenas uma das duas opções — decisão registrada antes de qualquer linha de código ser escrita, para evitar retrabalho de arquitetura.
- **v2** — Estrutura de 4 telas (`select` → `front` → `back` → `confirm`) controlada por `goTo(nome)`, que alterna a classe `.active` entre `<section>s` e decide quando iniciar/parar a câmera, com base nas referências de tela escura de captura (cantos-guia brancos, botão obturador central, ícone de flash).
- **v3** — Generalização da lógica de captura em `initCapture(side)`: em vez de duplicar o JavaScript para frente e verso, a mesma função é chamada duas vezes (`initCapture("front")` e `initCapture("back")`), cada uma operando sobre o DOM da sua própria seção — reduz divergência de comportamento entre as duas etapas.
- **v4** — Adição do alternador **"Tirar foto" / "Anexar arquivo"** (`.mode-toggle`) dentro de cada etapa de captura, com dropzone de arrastar-e-soltar como alternativa equivalente à câmera, incluindo pré-visualização de imagem quando o arquivo anexado é do tipo `image/*`.
- **v5** — Tratamento de falha de permissão/ausência de câmera: `startStream()` captura o erro do `getUserMedia` em `catch` e substitui o vídeo por uma mensagem orientando o uso do modo de anexo, evitando que o fluxo trave em dispositivos/navegadores sem acesso à câmera.

3. Responsividade

O container `.app` é um cartão único de largura máxima 430px, centralizado sobre um fundo em gradiente verde escuro — layout mobile-first por natureza, já que a etapa de captura de documento é primariamente um caso de uso de celular (câmera traseira). Diferente da tela de recuperação de senha, **não há um segundo layout de grid para desktop**: em telas largas o cartão simplesmente permanece centralizado com espaço vazio nas laterais, sem reorganização de colunas, pois o fluxo de captura não se beneficia de uma coluna

auxiliar de conteúdo institucional.

Contexto	Comportamento
Qualquer largura de viewport	Cartão único max-width: 430px, centralizado
Vídeo/preview da câmera	object-fit: cover dentro de .viewport-wrap, preenchendo a área disponível sem distorcer proporção
Dropzone de upload	min-height: 300px, mesma área ocupada pelo visor de câmera, para não haver "salto" de layout ao alternar entre os dois modos

4. Verificação da frente do documento

A etapa de frente (#screen-front) é a primeira etapa de captura após a seleção do tipo de documento (CNH ou RG) e segue a seguinte lógica client-side:

- Inicialização** — ao entrar na tela, `initCapture("front")` é chamada e o modo padrão é "camera", disparando `startStream("front", ...)`, que solicita `navigator.mediaDevices.getUserMedia({ video: { facingMode: "environment" } })` (preferência pela câmera traseira, quando disponível).
- Enquadramento** — quatro elementos `.corner` (cantos em L, brancos) são sobrepostos ao `<video>` via `position: absolute`, reproduzindo o guia visual de enquadramento das referências de câmera fornecidas, sem envolver nenhuma lógica de detecção real de bordas do documento (é puramente visual/orientativo).
- Captura** — o botão obturador (`[data-shutter]`) desenha o frame atual do `<video>` em um `<canvas>` oculto (`canvas.width/height = video.videoWidth/videoHeight`) e converte o resultado em `dataURL` via `canvas.toDataURL("image/jpeg", 0.9)`.
- Revisão obrigatória** — após a captura, a imagem é exibida em tela cheia com dois botões: **"Tirar novamente"** (reinicia o stream de vídeo) e **"Usar esta foto"** (persiste o `dataURL` em `state.images.front` e avança para a etapa de verso). Não há avanço automático — o usuário sempre confirma a foto antes de prosseguir.
- Modo alternativo (anexo)** — ao trocar para "Anexar arquivo", a câmera é interrompida (`stopStream("front")`) e um `<input type="file" accept="image/*,application/pdf">` assume o lugar do vídeo, aceitando também arraste-e-solte (`dragover/drop`); o arquivo é lido via `FileReader.readAsDataURL()` e passa pela mesma etapa de revisão antes de ser confirmado.

5. Verificação do verso do documento

A etapa de verso (#screen-back) reutiliza **exatamente a mesma função** `initCapture("back")`, com três diferenças de comportamento em relação à etapa de frente:

- Indicador de progresso** — a barra de `.step-dots` exibe a primeira bolinha como `.done` (etapa concluída) e a segunda como `.current`, deixando explícito ao usuário que a frente já foi capturada e ele está na segunda de três etapas.
- Botão "Voltar"** — direciona para `#screen-front` (`data-back="front"`) em vez de para a seleção de documento, permitindo corrigir a foto da frente sem reiniciar todo o fluxo (o `dataURL` de `state.images.front` permanece salvo em memória durante essa navegação).
- Destino após confirmação** — o botão **"Usar esta foto"** desta etapa persiste o resultado em `state.images.back` e chama `goTo("confirm")`, em vez de avançar para outra etapa de captura — ou seja, a máquina de estados sabe, através do parâmetro `side` passado a `initCapture()`, se deve encadear

para a próxima captura ou para a tela de confirmação.

Fora esses três pontos, a verificação do verso segue **a mesma lógica de câmera, cantos-guia, revisão obrigatória e modo de anexo alternativo** descrita na seção 4, sem duplicação de código — é a mesma função JavaScript operando sobre um segundo `<section>` no DOM.

6. Envio e salvamento do documento

Nesta camada (somente front-end, sem back-end conectado), **não existe envio de rede nem persistência real** — o comportamento atual é o seguinte:

- **Armazenamento em memória** — as duas imagens confirmadas (frente e verso) ficam guardadas como **strings** `dataURL` (base64) no objeto de estado `state.images = { front: null, back: null }`, mantido em escopo de módulo no JavaScript da página.
- **Sem `localStorage/sessionStorage`** — propositalmente não é usado armazenamento do navegador, para não reter dados sensíveis (imagem de documento de identidade) além da sessão ativa da aba.
- **Exibição na confirmação** — ao chegar em `#screen-confirm`, a função `fillConfirm()` apenas atribui os `dataURL` já capturados às tags `<img>` de miniatura (`#thumbFront`, `#thumbBack`) — não há novo processamento, compressão ou upload nesse momento.
- **Botão "Continuar para o app"** — atualmente dispara somente um `alert()` de exemplo ("Prosseguindo para a próxima etapa do cadastro."), como placeholder explícito para o ponto de integração futura.
- **Perda de dados ao recarregar/fechar** — como não há persistência em disco, cliente ou servidor, as fotos capturadas são descartadas se a página for recarregada ou fechada antes da confirmação — comportamento aceitável para um protótipo de front-end, mas que **precisa ser substituído** antes de uso em produção.
- **Integração pendente (fora desta camada)** — o envio real das imagens (ex.: `FormData + fetch()`/`multipart` para um endpoint de upload, ou conversão do `dataURL` para `Blob` antes do envio), a validação server-side de tipo/tamanho de arquivo, o armazenamento seguro (storage criptografado / bucket com controle de acesso) e a associação do documento ao cadastro do usuário não estão implementados — assim como nas telas de login e recuperação de senha, este relatório documenta apenas a camada client-side.

7. Acessibilidade (WCAG-aligned)

- Botões de navegação (voltar, obturador, flash, alternância de modo) com `aria-label` ou texto visível suficiente para leitura por leitor de tela.
- Imagens de pré-visualização (`<img class="captured-preview">`, `<img class="dz-preview">`) com atributo `alt` descritivo específico por lado do documento ("Foto capturada da frente do documento" / "...do verso do documento").
- Indicador de progresso (`.step-dots`) reforçado textualmente pelo título de cada etapa ("Frente do documento" / "Verso do documento"), não dependendo apenas da cor das bolinhas para comunicar o avanço.
- Mensagem de fallback de câmera (`.camera-off-msg`) escrita em texto simples e direto, orientando explicitamente o uso do modo de anexo quando a câmera não está disponível — evita que o usuário fique bloqueado sem explicação.
- Área de toque do botão obturador dimensionada (66px) acima do mínimo recomendado para uso confortável em touchscreen.

8. Organização de assets

Assim como na tela de histórico de consultas, a logomarca é um **SVG inline** (mesmo gradiente turquesa/verde), sem dependência de arquivo de imagem externo. Todos os ícones de interface (voltar, flash,

upload, sucesso, documento CNH/RG, cadeado) também são SVGs inline, embutidos diretamente na marcação HTML — nenhum ícone de terceiros (ex.: bibliotecas de ícone via CDN) foi utilizado.

9. Testes executados

Verificação manual do fluxo completo (seleção → frente → verso → confirmação) nos dois modos de captura (câmera e anexo), incluindo o cenário de permissão de câmera negada/indisponível (fallback textual exibido corretamente) e o cenário de arraste-e-solte de arquivo na dropzone. Não há suíte de testes automatizados (Playwright/Jest) implementada nesta rodada, e não há testes de integração com back-end real, já que nenhuma rota de API está conectada nesta camada.

10. Fora de escopo

Envio real das imagens para um servidor, validação server-side de tipo/tamanho/qualidade do documento, verificação automática de autenticidade ou legibilidade do documento (OCR, detecção de bordas, checagem de vencimento), armazenamento seguro e persistente das imagens, expiração de sessão de verificação e a etapa de biometria/confirmação de idade mencionada no material de referência (VERIFICAÇÃO_CONCLUIDA.pdf) não estão implementados nesta tela — o escopo atual cobre exclusivamente a captura e a revisão local das fotos de frente e verso. Essas integrações deverão ser cobertas em relatório próprio quando a camada de back-end for conectada.